

# AI Museum of the Future

## Weekly Validation Report: Corpus, Chunking, Retrieval, Agents, Prompts and Multimodal Components

Esprit 2026-2027, 5DS, Advanced Deep Learning (AI Automation Final Project). October 6, 2026.

### Summary

This report documents the validation of six milestone items for the retrieval-augmented generation and multi-agent pipeline developed this week. Rather than relying on unit testing alone, the team validated the complete system by running it end to end against production infrastructure, specifically a live Supabase vector database and the OpenAI API. This approach revealed seven integration defects that unit tests could not have detected, since those tests mock the very interfaces at which the defects occurred. Each item discussed below is therefore supported by evidence obtained through direct execution of the system, rather than by design intent alone.

| Milestone item                            | Status   | Evidence                                                                                                                 |
|-------------------------------------------|----------|--------------------------------------------------------------------------------------------------------------------------|
| Corpus collected                          | Complete | 2,767 documents and 2,777 chunks held in Supabase, contributed by three independent pipelines across ten or more sources |
| Chunking strategy implemented             | Complete | Fixed-size and recursive strategies, both covered by property-based tests                                                |
| Initial retrieval tests completed         | Complete | Five queries executed against the live corpus, discussed in Section 3                                                    |
| Agents implemented and communicating      | Complete | Five specialist agents and a curator agent produced a complete, grounded exhibition during a live run                    |
| Prompts V1 defined and tested             | Complete | Versioned prompt library, including a documented revision described in Section 5                                         |
| Multimodal component working in isolation | Complete | A real photograph processed into a structured description and an embedding using GPT-4o vision                           |

### 1. Corpus Collection

Two complementary data collection pipelines were developed, each drawing exclusively on official public application programming interfaces; no content was obtained through web scraping.

The first pipeline focuses on Tunisia and draws on six sources: GDELT, Wikipedia (through both the pageviews endpoint and REST article summaries), Wikidata (queried via SPARQL), Wikimedia Commons, arXiv, and the World Bank Open Data API. This pipeline produced 2,613 normalized records spanning the period from 2020 to 2026.

The second pipeline collects material across a broader set of global and topical sources, drawing on seven APIs: GDELT, arXiv, the Metropolitan Museum of Art Open Access API, Openverse, the Internet Archive, DBpedia, and a trends feed derived from GDELT article volume. It spans thirty-six countries, including all twenty-two Arab League member states, and more than twenty-two topics ranging from artificial intelligence and climate change to culture, the economy, sport, and the 2026 FIFA World Cup. This pipeline yielded over 3,750 records and more than 700 downloaded images.

A third contribution to the shared corpus came from the team's n8n automation workflow, which satisfies the project's automation platform requirement. This workflow independently ingested 153 documents from arXiv, NewsAPI, and BBC and Reuters open data feeds.

The selection rationale for each source, together with known limitations such as rate limiting, coverage gaps, and formatting inconsistencies, is documented in `data/README.md`. Taken together, the three contributions are reflected in the shared Supabase vector store, which at the time of writing holds 2,767 documents and 2,777 chunks, combined without data loss or duplication.

## 2. Chunking Strategy

The module `rag/chunking/chunker.py` implements two chunking strategies and allows either to be selected at runtime, as required by the assignment. The fixed-size strategy slides a token window, tokenized with `tiktoken`, across the input text with configurable overlap. The recursive strategy splits text first on paragraph boundaries, then on lines, falling back to fixed-size splitting for fragments that remain too large.

Both strategies are covered by unit tests and by a property-based test, implemented with Hypothesis, that verifies non-empty output is produced for any non-empty input text across one hundred randomly generated examples per run.

## 3. Retrieval Evaluation

The module `rag/vector_store/retriever.py` embeds an incoming query using OpenAI's `text-embedding-3-small` model, issues a `match_chunks` remote procedure call to Postgres that performs cosine similarity search through `pgvector`, and returns ranked results with an optional reranking pass using `gpt-4o`. Five queries were executed against the complete, live corpus to evaluate retrieval quality.

| Query                                | Top similarity | Top result (abridged)                                             |
|--------------------------------------|----------------|-------------------------------------------------------------------|
| key AI breakthroughs in 2026         | 0.591          | "The rise of AI over the last 8 decades..." (Wikimedia)           |
| Tunisia economy and unemployment     | 0.390          | GDP per capita indicator for the North Africa region (World Bank) |
| climate change protests and activism | 0.467          | "Street art in Sydney. Climate change. 2026" (Wikimedia)          |
| contemporary art in North Africa     | 0.546          | "Zeitz Museum of Contemporary Art Africa, Cape Town" (Wikimedia)  |
| space exploration milestones         | 0.285          | Relevant arXiv research on robotics and world models              |

*A genuine defect was identified while preparing this evaluation. The vector index built on the chunks table, an ivfflat index from `pgvector`, had been constructed with parameters unsuited to the table's row count, with the effect that it silently returned zero results for every query and raised no error. This is a documented `pgvector` pitfall. The index was diagnosed and rebuilt with corrected parameters during this session, after which retrieval returned the consistently relevant results shown above.*

## 4. Multi-Agent Synthesis

Five specialist agents, namely Historian, Sociologist, Technology, Culture and Art, and Visual, independently analyze the same retrieved evidence from complementary perspectives, sharing their findings through a cross-reference layer. A curator agent then synthesizes the five perspectives into a structured exhibition consisting of a title, a concept statement, between four and six narrative sections, and

a list of selected artifacts.

A complete live run of the pipeline, executed through `test_agents_live.py`, produced an exhibition titled “Voices of Change: Technology, Culture, and Activism in 2026,” comprising five grounded sections (Advancements in Artificial Intelligence, Quantum Computing Milestones, Global Climate Activism, Cultural Expressions of Activism, and Visual Language of Change) together with a list of selected artifacts drawn directly from the retrieved evidence.

## 5. Prompt Engineering

A versioned prompt library, held in `prompts/*.md`, covers all five specialist agents together with the curator agent, and is evaluated through `prompts/evaluate_prompts.py`. A concrete revision was carried out and documented during this session.

In its initial form, the curator prompt's example output showed only two sections and did not state the system's actual requirement of four to six sections anywhere in its instructions. During the live run, the pipeline failed validation with the error “curator output must include four to six sections,” as the model had mirrored the structure of the example rather than the constraint enforced by the code.

The revised prompt states the requirement explicitly within the instructions and extends the example to five placeholder sections rather than two. When re-run against the identical input that had previously failed, the revised prompt produced a valid result immediately.

## 6. Multimodal Analysis

The module `multimodal/analyze_image.py` submits an image to GPT-4o vision and extracts a structured description consisting of detected objects, a scene summary, a free-text description, and thematic tags, together with a 1536-dimension embedding intended for later retrieval. The component was tested against real photographs drawn from the collected corpus, for example a news image obtained through GDELT, and produced accurate, well-structured output.

*A further defect was identified and corrected in this component. The original parsing logic assumed that the model would respond using a flat label-and-value format, whereas the model in practice replies using Markdown headings such as “### Objects.” This mismatch caused the parser to raise an exception, which was silently caught and replaced with a generic fallback message, discarding a perfectly usable description in the process. The parsing logic was rewritten to extract fields defensively, without raising, regardless of the exact formatting the model chooses to use.*

## 7. Implementation Challenges and Resolutions

Running the complete pipeline against production infrastructure, rather than treating a passing unit test suite of 247 tests as sufficient evidence of correctness, surfaced seven integration defects that mocked tests could not have revealed. Each was diagnosed, corrected, verified, and committed within the same session, and is summarized in Table 2.

| No. | Observed symptom                                            | Root cause                                                                                           | Resolution                                                                         |
|-----|-------------------------------------------------------------|------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------|
| 1   | Retrieval silently returned zero rows, with no error raised | The ivfflat vector index had been built with parameters unsuited to the table's row count            | The index was rebuilt with corrected parameters and now returns relevant results   |
| 2   | Document insertion failed with a schema error               | The application code assumed a <code>source_id</code> column that had never been added to the schema | The column was added, together with a unique index to support proper deduplication |

|   |                                                                                 |                                                                                  |                                                                                                       |
|---|---------------------------------------------------------------------------------|----------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------|
| 3 | Re-running ingestion either failed or produced duplicate rows                   | The upsert operations did not specify a conflict target                          | An explicit on_conflict clause was added to both the document and chunk upsert operations             |
| 4 | Ingestion failed on a single Wikidata record                                    | A bare-year date value such as "2013" is rejected outright by a timestamp column | A defensive date parser was introduced; values that cannot be parsed are retained in metadata instead |
| 5 | A multi-thousand-record ingestion run terminated after one network interruption | No retry logic existed around the database calls                                 | A retry mechanism with backoff was added around both upsert operations                                |
| 6 | The curator agent failed output validation                                      | The prompt did not state its own requirement of four to six sections             | The prompt was revised, as described in Section 5                                                     |
| 7 | Multimodal analysis silently returned "image unavailable"                       | The parser failed on the model's actual Markdown-formatted response              | The extraction logic was rewritten to be defensive and non-raising                                    |

*Each resolution was verified against its originally failing case and against the complete automated test suite before being committed and pushed to the shared repository.*

## 8. Current State and Future Work

At the time of writing, the automated test suite passes in full, with 247 of 247 tests succeeding across unit and property-based categories. The shared Supabase project holds 2,767 documents and 2,777 chunks, combining the contributions of all three pipelines described in Section 1. Three scripts were prepared for live demonstration during validation: `demo_retrieval.py`, `test_agents_live.py`, and `demo_multimodal.py`.

One limitation remains open and is reported here rather than concealed. Three team members independently developed data pipelines with some overlap in scope, namely the global corpus pipeline and two separate Tunisia-focused pipelines. These have not yet been reconciled into a single canonical pipeline. Resolving this is planned ahead of the final report, consistent with the project's stated preference for a simple architecture that the team can defend over a more complex one it cannot fully explain.